



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING - CAPSTONE PROJECT REPORT

A CAPSTONE PROJECT REPORT

Secure Digital Voting and Election Management System with Authentication Using Object-Oriented Programming in C++

A Capstone Project Report submitted in partial fulfilment of the requirements for the course of

DSA-0112 Object-oriented Programming in C++

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

Submitted by

CH. Venkata Chandu (192424140)

Under the Supervision of

Dr. I. Sudha

Professor / CSE, SIMATS ENGINEERING

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "Secure Digital Voting and Election Management System with Authentication Using Object-Oriented Programming in C++" has been carried out by CH. Venkata Chandu (192424140), R.V Sreenivasan (192521417), and S.Sanjay (192521058) under the supervision of Dr. I. Sudha and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech Artificial Intelligence and Data Science program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr. C. Sivasankar
Professor, Department of CSE
SIMATS Engineering
SIMATS, Chennai - 602105.

COURSE FACULTY

Dr. I. Sudha
Professor, Department of CSE
SIMATS Engineering
SIMATS, Chennai - 602105.

DECLARATION

I, CH. Venkata Chandu, of the Department of Artificial Intelligence and Data Science, SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Secure Digital Voting and Election Management System with Authentication Using Object-Oriented Programming in C++" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 03.09.2026

CH. Venkata chandu

Signature of the Student with name

INDIVIDUAL CONTRIBUTION STATEMENT

Student / Register No.	Specific Responsibilities	Design & Development	Testing & Analysis	Report Contribution	Approx. %
CH. Venkata Chandu 192424140	System architecture, web routing and integration	Designed backend routing and C++ server integration	API endpoint and routing validation	Chapters 1 and 3	34%
R.V Sreenivasan 192521417	Authentication and registration	Developed Auth Manager, User/Voter model and file I/O logic	Credential and age- validation testing	Chapters 2 and 4	33%
S. Sanjay 192521058	Voting process and result handling	Developed Election Manager, Candidate vector and tallying logic	Duplicate-vote prevention tests	Chapter 5 and report formatting	33%
Total					100%

The contribution percentages indicate the approximate share of technical design, implementation, testing, documentation, and integration effort performed by each team member. The work was reviewed collectively before final integration.

ABSTRACT

Traditional paper ballot systems may suffer from counting errors, processing delays, and difficulties in verifying every voter consistently. This creates a need for a software-based election management approach that can authenticate voters, record ballots accurately, prevent repeated submissions, and generate results rapidly. The proposed project develops a Secure Digital Voting and Election Management System using Object-Oriented Programming in C++.

The implementation is organized into three main modules: User Authentication, Voter Registration Management, and Voting Process Management. Encapsulation is used to protect authentication and voter state, inheritance models common user information through a Voter class derived from User, and C++ STL containers such as `std::vector` manage candidate records and vote tallies. File-stream operations using `std::fstream` provide local persistence without requiring an external database, while a lightweight HTTP routing layer connects the C++ backend with the web interface.

Implementation evidence includes voter login, new voter registration, candidate selection, successful vote submission, and duplicate-vote rejection screens. Verification tests focus on age validation, voter authentication, successful vote casting, and prevention of repeated voting. The resulting prototype demonstrates a practical, lightweight election workflow with faster processing and automatic tallying compared with manual vote counting. The report also identifies limitations such as local plain-text persistence and the absence of cryptographic credential storage, which are addressed as future improvements.

Keywords: C++, Object-Oriented Programming, Digital Voting, Voter Authentication, File Persistence, Election Management.

TABLE OF CONTENTS

Sl. No.	Title	Page No.
1	CERTIFICATE FROM THE SUPERVISOR	i
2	DECLARATION BY THE CANDIDATE	ii
3	INDIVIDUAL CONTRIBUTION STATEMENT	iii
4	ABSTRACT	iv
5	LIST OF FIGURES	v
6	LIST OF TABLES	vi
7	LIST OF ABBREVIATIONS / SYMBOLS	vii
8	CHAPTER 1: Introduction and Engineering Problem	1
9	CHAPTER 2: Literature Review and Existing Solutions	2
10	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	3
11	CHAPTER 4: Implementation, Testing & Results, Project Management	4
12	CHAPTER 5: Conclusion, Future Work and Reflection	5
13	References	
14	Appendices	

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 3.1	Secure Digital Voting System Architecture	11
Figure 4.1	Voter Login Interface	15
Figure 4.2	New Voter Registration Interface	15
Figure 4.3	Candidate Selection Interface	16
Figure 4.4	Successful Vote Submission	16
Figure 4.5	Duplicate Vote Rejection	17
Figure 4.6	Project Demonstration / Final Implementation Evidence	17

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1.1	Project objectives	9
Table 2.1	Comparative analysis of existing approaches	10
Table 3.1	Functional and non-functional requirements	12
Table 3.2	Constraints and applicable standards	12
Table 3.3	Design specifications	13
Table 3.4	Alternative solution evaluation	13
Table 3.5	Trade-off analysis	14
Table 3.6	Sustainability, ethics, safety and security	14
Table 3.7	Risk register	14
Table 4.1	Software and tools used	15
Table 4.2	Test plan and verification	17
Table 4.3	Achievement of objectives	18
Table 4.4	Team roles and actual contribution	18
Table 4.5	Indicative project cost	19

LIST OF ABBREVIATIONS / SYMBOLS

Symbol / Abbreviation	Description	Unit
C++	C Plus Plus programming language	-
OOP	Object-Oriented Programming	-
STL	Standard Template Library	-
HTTP	Hypertext Transfer Protocol	-
API	Application Programming Interface	-
EVM	Electronic Voting Machine	-
UI	User Interface	-
ID	Identifier	-
I/O	Input / Output	-
JSON	JavaScript Object Notation	-
ms	Millisecond	ms

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Elections depend on accurate voter identification, controlled ballot access, reliable vote recording, and correct result generation. Traditional paper ballot systems are familiar and direct, but they require physical materials, manual verification, manual counting, and careful handling of records. These activities create opportunities for delays and human error when the number of voters increases.

The proposed project explores a software-oriented approach using C++ and object-oriented design. A centralized application receives voter information, verifies credentials, manages eligible voters, records a ballot only after authentication, and updates the candidate tally. The design keeps the core election rules inside the backend rather than relying entirely on the browser interface.

1.2 Need for the Project

The system is needed to reduce repetitive manual work in a controlled voting environment such as a college-level election, club election, or demonstration election. The main engineering need is to establish a clear sequence: registration, authentication, voting, and result generation. The design also needs to detect repeated attempts to vote and preserve voter information across server restarts.

- Manual verification consumes time and can be inconsistent across voters.
- Duplicate voting must be prevented through backend state validation.
- Manual counting introduces avoidable arithmetic and transcription errors.
- A software workflow can provide faster tally generation and clearer audit evidence.

1.3 Problem Statement

Manual voting takes more time, makes it difficult to verify every voter consistently, may allow a voter to attempt more than one ballot, and can introduce mistakes during manual counting. A digital election management system is therefore required to authenticate eligible voters, restrict ballot access, record one valid vote per voter, and generate accurate results efficiently.

1.4 Project Aim

To develop a secure digital voting and election management system using C++ and Object-Oriented Programming, with voter authentication, duplicate-vote prevention, persistent records, and automated vote counting.

1.5 Project Objectives

1. To design a modular voting system using C++ and OOP principles.
2. To authenticate registered voters before granting access to the ballot.
3. To validate voter registration and enforce a minimum legal voting age of 18 for the prototype.
4. To prevent duplicate voting using backend voter-state validation.
5. To store voter and candidate information using file-based persistence.
6. To provide fast and accurate result generation through automatic tallying.
7. To test the system using positive and negative functional test cases.

Table 1.1 Project Objectives

Objective	Measurable Evidence
OBJ-1	To design a modular voting system using C++ and OOP principles.
OBJ-2	To authenticate registered voters before granting access to the ballot.
OBJ-3	To validate voter registration and enforce a minimum legal voting age of 18 for the prototype.
OBJ-4	To prevent duplicate voting using backend voter-state validation.
OBJ-5	To store voter and candidate information using file-based persistence.
OBJ-6	To provide fast and accurate result generation through automatic tallying.
OBJ-7	To test the system using positive and negative functional test cases.

1.6 Scope

- Included: voter registration, authentication, ballot casting, duplicate-vote rejection, candidate management support, result tallying, and local persistence.
- Excluded: biometric authentication, national-scale distributed infrastructure, cryptographic election protocols, and production-grade database clustering.
- Assumption: the prototype runs in a controlled server environment where local storage and application files are protected by the host system.

1.7 Expected Engineering Outcome

The expected outcome is a functional software prototype consisting of a C++ backend, a lightweight HTTP interface, a web-based user interface, module-level validation logic, and documented implementation evidence. The system should demonstrate the complete workflow from voter registration through authentication, voting, result processing, and rejection of a duplicate ballot.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review compares the project concept with common voting approaches described in the project materials: manual paper ballots and conventional electronic voting machines. The comparison focuses on deployment requirements, counting speed, verification effort, accessibility, and relevance to a lightweight academic software prototype.

2.2 Review of Existing Work

Paper Ballot Systems

Paper ballots are easy to understand and do not require end-user software, but they depend on physical polling infrastructure. Voter verification and counting are largely manual, which increases the time required to complete an election and can create opportunities for counting or handling errors.

Electronic Voting Machines

Electronic voting machines can accelerate vote recording and counting, but they rely on specialized hardware and controlled deployment environments. Their hardware dependence makes them less suitable as the basis for a software-only academic prototype intended to run on standard computing equipment.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual Paper Ballot	Simple process; familiar to users	Manual verification, counting effort, physical record handling	Shows need for digital automation and faster tallying
Traditional EVM	Fast electronic counting; dedicated interface	Specialized hardware; limited flexibility for a software-only prototype	Motivates a lightweight C++ software implementation
Proposed C++ System	Modular OOP design; authentication; automatic tally; local persistence	Prototype scale; plain-text local storage; limited security hardening	Provides the project-specific architecture and implementation platform

2.4 Research / Engineering Gap

The engineering gap addressed by the project is the need for a lightweight election management prototype that can demonstrate core security and voting rules without depending on expensive hardware or a full external database stack. The project focuses on software structure, backend enforcement of rules, and practical evidence of implementation.

2.5 Proposed Contribution

The proposed contribution is an object-oriented C++ election management system that combines voter authentication, inheritance-based data modeling, vector-based candidate management, file persistence, HTTP routing, and duplicate-vote prevention. The core business rules are enforced by backend objects and manager classes so that the browser UI cannot bypass them simply by changing its display state.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Voter	Register with valid details, authenticate, and cast one ballot.
Election Administrator	Manage voters/candidates and inspect results.
Project Assessor	Verify implementation logic, testing evidence, and documentation.

Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Age Verification	Functional	Reject registration when age < 18.
Unique Identity	Functional	Reject an already-registered voter ID.
Duplicate Vote Guard	Functional	Reject ballot when hasVoted == true.
Authentication	Functional	Do not allow voting access without valid credentials.
Persistence	Functional	Write registered voter/candidate state to local files.
OOP Structure	Non-Functional	Use classes, encapsulation, inheritance and STL.
Response Time	Non-Functional	Prototype API processing target below 100 ms under normal local use.

3.2 Constraints & Applicable Standards

Standard / Code	Requirement	How Applied in This Project
HTTP/1.1	Web communication	Lightweight C++ HTTP routing is used for login, registration, voting and results endpoints.
C++17	Language standard	std::string, std::vector, std::fstream, lambdas and class-based OOP are used in the implementation.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Response Time	API processing < 100	ms	High
External Database	None for prototype	Count	High
Authentication	Required before voting	Boolean	High
Duplicate Vote State	One vote per registered voter	Boolean	High
Minimum Registration Age	18	years	High
Persistent Storage	Local voter and candidate records	Files	Medium

3.4 Alternative Solutions & Evaluation

Alternative 1 - External SQL Database: provides stronger query and concurrency capabilities but adds database setup, dependency, and configuration complexity.

Alternative 2 - In-memory arrays only: provides very simple and fast access but loses voter and candidate state when the server restarts.

Alternative 3 - C++ fstream text storage: provides portable local persistence with no database server requirement, making it suitable for the academic prototype.

Criterion	Weight	SQL Database	In-memory Arrays	fstream Files
Performance	0.25	5	5	4
Cost / Setup	0.20	3	5	5

Persistence	0.20	5	1	4
Portability	0.15	3	5	5
Prototype Simplicity	0.20	3	5	5

3.5 Selected Design & Detailed Design

The selected design uses a modular C++ backend with three primary functional modules. The User Authentication Module validates credentials and controls access to voting functions. The Voter Registration Management Module validates new voter information, models voter data through inheritance, and persists records to local files. The Voting Process Management Module uses candidate objects and a duplicate-vote flag to ensure a single valid ballot per voter.

Secure Digital Voting and Election Management System Architecture

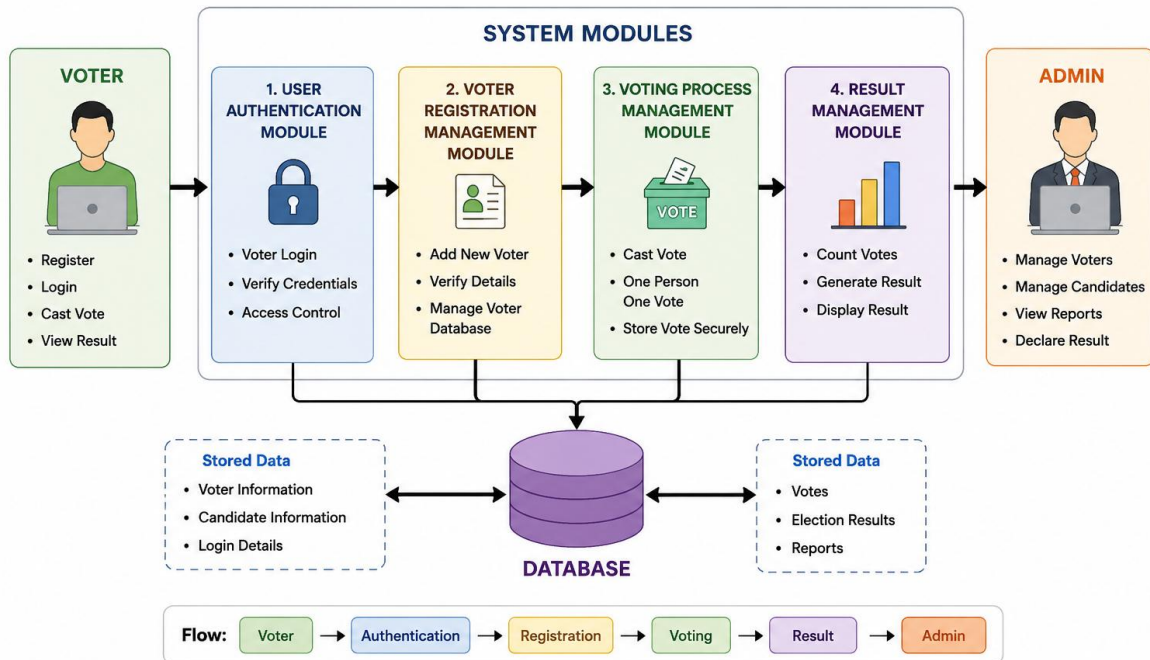


Figure 3.1 Secure Digital Voting System Architecture (implementation architecture from the project presentation)

Subsystem interfaces follow a controlled sequence: the voter first authenticates, then enters the voting interface; the voting module validates voter state and candidate selection; the persistence layer stores updated records; and the result module reads the latest tally for display. This separation keeps authentication and vote integrity decisions in the backend.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
File I/O Storage	MySQL / external SQL	Zero database dependency; simple deployment	Lower scalability and weaker security controls	Selected for a small academic prototype where portability is more important than high-scale concurrency.
Backend duplicate-vote flag	Frontend-only validation	Cannot be bypassed by normal UI changes	Requires server-side state management	Selected because vote integrity must be enforced by trusted backend logic.
OOP class structure	Single large procedural program	Improves separation of responsibilities and reusability	More code structure required	Selected because the course emphasizes OOP concepts.

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Avoid unnecessary infrastructure	Prototype uses standard computer hardware and local files	Reduced dependency on dedicated election hardware.
Ethics	One-person-one-vote integrity	Duplicate-vote state check and authentication	Backend rejection is required before the ballot is accepted.
Health & Safety	Safe software-only interaction	No physical actuation or hazardous hardware	Normal computer usage only.
Security	Unauthorized access and credential exposure	Authentication gate and backend validation	Voting endpoints must reject unauthenticated requests.

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Duplicate voting attempt	Medium	High	High	Check hasVoted before accepting ballot and update state after success.	Low
Unauthorized access	Medium	High	High	Validate credentials before voting and return unauthorized response.	Medium
Local file modification	Low	High	Medium	Restrict host file access and consider cryptographic storage in future.	Medium
Server restart / data corruption	Low	Medium	Low	Persist records to files and maintain backups during testing.	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project was developed as a modular C++ software prototype. The backend contains classes for users, voters, candidates, authentication, election control, and persistence. A lightweight HTTP routing layer exposes application functions to the web interface, while HTML/JavaScript screens provide the voter-facing interaction. The development workflow focused on implementing each module, connecting the endpoints, running functional checks, and recording screenshots as implementation evidence.

4.2 Software Implementation & Modern Tools Used

Tool / Software	Purpose	Stage Used
C++17 / MinGW g++	Core backend logic, OOP classes, compilation	Development & compilation
cpp-httpplib	HTTP server and API routing	Integration
CMake	Build configuration	Compilation / project setup
HTML / CSS / JavaScript	User interface screens	Front-end integration
std::fstream	Local persistence for voter/candidate records	Data storage
Visual Studio Code	Source editing and project management	Development / debugging

Implementation

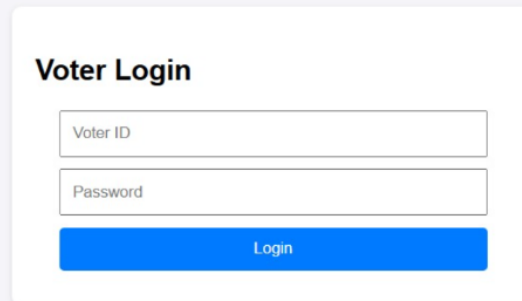
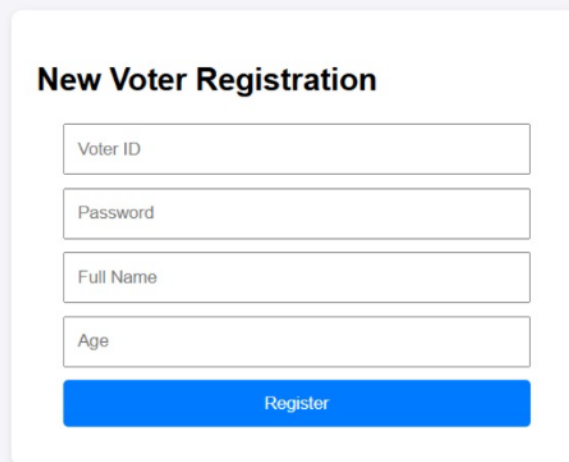


Figure 4.1 Voter Login Interface



The login screen captures the voter identifier and password before the backend authentication route is called. The registration screen captures voter ID, password, full name, and age. The backend validates the age and identifier uniqueness before creating the voter object.

4.2 Software Implementation - Voting Interface

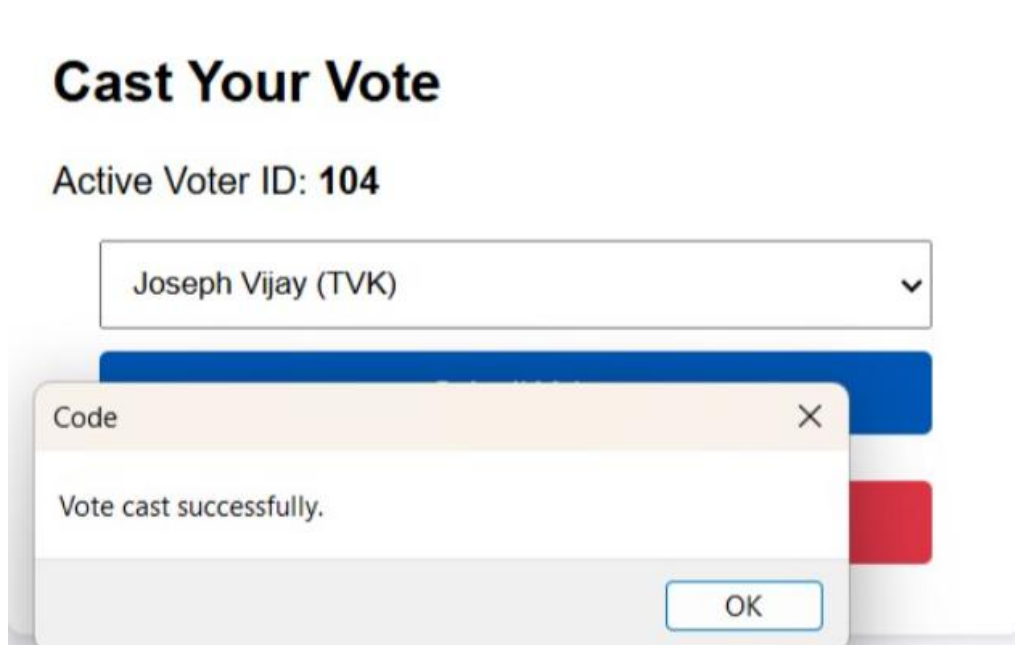


Figure 4.3 Candidate Selection and Successful Vote Submission Interface

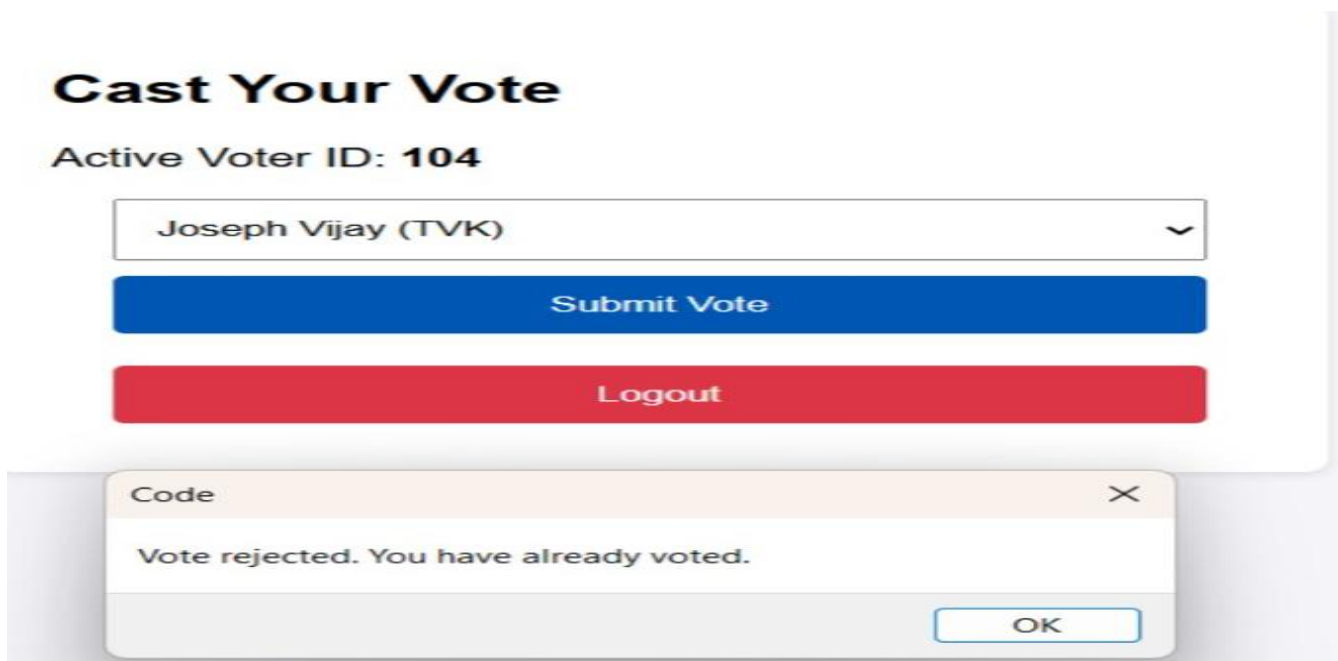


Figure 4.4 Duplicate Vote Rejection Interface

The voting interface demonstrates that a voter selects a candidate and submits the ballot. On a first valid submission, the backend increases the selected candidate vote count and marks the voter as having voted. A second submission for the same active voter is rejected rather than incrementing the candidate count again.

4.3 Test Plan & Verification

Requirement	Test Method	Acceptance Criterion	Achieved Value	Status
Voter Registration	Enter age below 18	Registration rejected	Failure message shown	PASS
Unique Voter ID	Register an existing ID	Duplicate registration rejected	Existing ID blocked	PASS
Authentication	Use valid and invalid credentials	Only valid credentials accepted	Valid login accepted; invalid login rejected	PASS
Cast Ballot	Select candidate and submit	Vote recorded once	Success message shown	PASS
Prevent Duplicates	Submit second ballot using same voter	Second ballot rejected	Duplicate rejection shown	PASS
Result Tally	Read results after valid votes	Counts reflect accepted ballots	Candidate tally updated	PASS

4.4 Results

Functional testing demonstrated the intended workflow from voter authentication to vote submission. The implementation evidence shows a successful vote dialog after a valid ballot and a rejection dialog when the same voter attempts to vote again. The automatic count update removes the need for manual arithmetic during the prototype demonstration.

4.5 Data Analysis & Engineering Judgment

The strongest engineering observation is that vote integrity depends on backend state rather than visual controls in the user interface. The `hasVoted` flag is checked before the candidate tally is updated, and only then is the vote state persisted. This ordering avoids accepting a second ballot from the same authenticated voter in normal application use. The project also demonstrates the benefit of keeping credential checks inside `AuthManager` rather than duplicating the same checks across multiple routes.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Objective	Evidence	Achievement
Develop an OOP voting system	User, Voter, Candidate and manager classes	Fully Achieved
Authenticate voters before voting	Authentication route validates stored credentials	Fully Achieved
Prevent duplicate voting	<code>hasVoted</code> state blocks repeat submissions	Fully Achieved
Persistent records	<code>fstream</code> -based local storage	Fully Achieved
Fast and accurate result generation	Automatic candidate vote incrementing	Fully Achieved

4.7 Strengths & Limitations

Strengths

- Clear modular separation of authentication, registration and voting logic.
- Backend enforcement of duplicate-vote rules.
- Automatic tallying and immediate result preparation.
- Runs on standard computer hardware without an external database server.
- Provides direct visual evidence for key user interactions.

Limitations

- Local plain-text storage is not appropriate for high-security production elections.
- No cryptographic password hashing is currently included in the demonstrated prototype.
- The architecture is intended for a controlled academic-scale deployment rather than national-scale concurrent voting.
- Full audit logging, formal threat modeling, and independent security assessment are outside the current project scope.

4.8 Project Planning, Roles & Budget

Student	Assigned Responsibility	Actual Contribution
CH. Venkata Chandu	Routing & Web Layer	Built/connected HTTP server routes and system integration
R.V Sreenivasan	Registration & Authentication	Implemented AuthManager, voter model and file parsing/persistence
S.Sanjay	Election Logic	Implemented ElectionManager, candidate storage, tallying and duplicate guard

Indicative Project Budget

Item	Estimated Cost (INR)	Actual Cost (INR)
Existing laptop / desktop	0	0
Internet / connectivity for development	500	500
Software tools / compilers	0	0
Printing / documentation	0	0
Miscellaneous testing	0	0
Total	500	500

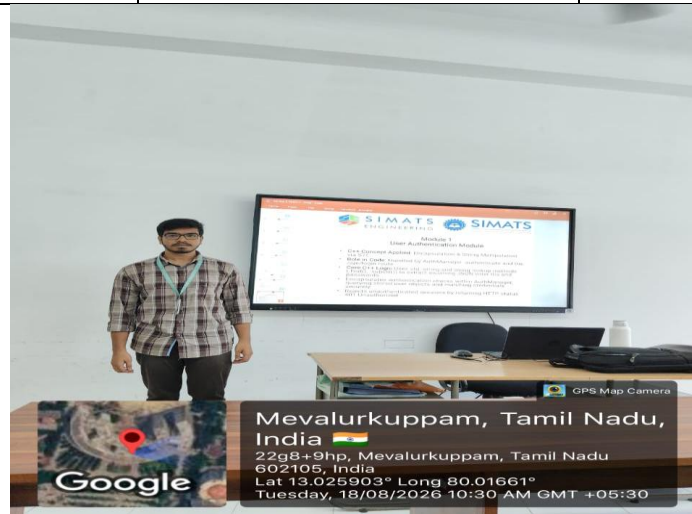


Figure 4.5 Project demonstration / final implementation evidence

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Secure Digital Voting and Election Management System demonstrates how C++ Object-Oriented Programming can be applied to an end-to-end election workflow. The project combines voter registration, authentication, ballot casting, duplicate-vote prevention, persistent records, and automatic result processing within a modular software architecture. Functional tests and implementation screenshots provide evidence that valid voters can access the ballot, that a valid vote is recorded, and that a second vote attempt is rejected.

The design is appropriate as an academic prototype because it emphasizes the engineering concepts required by the course while keeping the implementation compact and portable. The major contribution is the integration of OOP structure with practical election business rules. The project does not claim production election security; rather, it establishes a working foundation that can be strengthened through cryptography, stronger persistence, audit logging, and more rigorous deployment controls.

5.2 Future Work

- Store passwords using a modern password-hashing scheme rather than plain-text credentials.
- Replace local text files with a transactional database for larger datasets and stronger consistency.
- Add cryptographic protection for sensitive network communication and ballot-related data.
- Introduce role-based administration, audit logs, and configurable election parameters.
- Add stronger session management, lockout controls, and security monitoring.
- Evaluate concurrency, performance, and failure recovery with larger synthetic workloads.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Applying encapsulation, inheritance and STL containers to a complete application.
- Connecting C++ backend classes to HTTP routes and web UI interactions.
- Using fstream for persistent records and understanding its trade-offs.
- Designing negative test cases for authentication and duplicate-vote scenarios.

Engineering & Professional Skills Developed

- Modular system architecture and interface design.
- Debugging and endpoint validation.
- Collaborative code integration and documentation.
- Technical communication through diagrams, tables and implementation evidence.

Individual Reflection

CH. Venkata Chandu: The most difficult part was connecting backend routing with the user interface while preserving a clear request flow. The main engineering decision was to keep election rules in backend classes so that UI actions could not directly change vote state. I developed stronger understanding of HTTP routing and integration during implementation. If the project were repeated, I would add stronger authentication and centralized logging from the beginning.

R.V Sreenivasan: The most difficult part was implementing voter validation and persistence while keeping the User and Voter classes simple. The key engineering decision was to use inheritance for shared attributes and to enforce registration rules before creating a voter record. I gained practical understanding of class design, file streams and input validation. A future version should use secure credential storage and stronger data validation.

S.Sanjay: The main challenge was preventing duplicate voting while updating candidate totals correctly. I selected a hasVoted state check together with vector-based candidate storage because it made the voting logic straightforward and testable. I learned how backend state and UI evidence must agree during functional testing. A redesigned implementation would include database transactions and an auditable result history.

REFERENCES

- [1] cppreference.com, “C++ Standard Library Documentation,” Online documentation for C++ language and standard library features.
- [2] Y. Hirose, “cpp-httplib: A C++ header-only HTTP/HTTPS server and client library,” GitHub project documentation.
- [3] Project presentation, “Secure Digital Voting and Election Management System,” Review 2 team presentation, SIMATS Engineering, 2026.
- [4] Capstone project template, “A Capstone Project Report,” SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, 2026.
- [5] M. A. et al., “A Systematic Literature Review of Current Trends in Electronic Voting System Protection Using Modern Cryptography,” *Applied Sciences*, vol. 14, no. 7, 2024, Art. no. 2742.
- [6] “Approach based on STPA extended with STRIDE and LINDDUN, and blockchain to develop a mission-critical e-voting system,” *Journal of Information Security and Applications*, vol. 81, Mar. 2024, Art. no. 103715.
- [7] “E-voting system using cloud-based hybrid blockchain technology,” *Journal of Safety Science and Resilience*, vol. 5, no. 1, pp. 102–109, Mar. 2024.
- [8] L.-H. Merino, “Electronic Voting,” in *Trends in Data Protection and Encryption Technologies*, Springer, 2023.
- [9] S. J. Turnbull-Dugarte and D. Devine, “Support for digitising the ballot box: A systematic review of i-voting pilots and a conjoint experiment,” *Electoral Studies*, vol. 86, Dec. 2023, Art. no. 102679

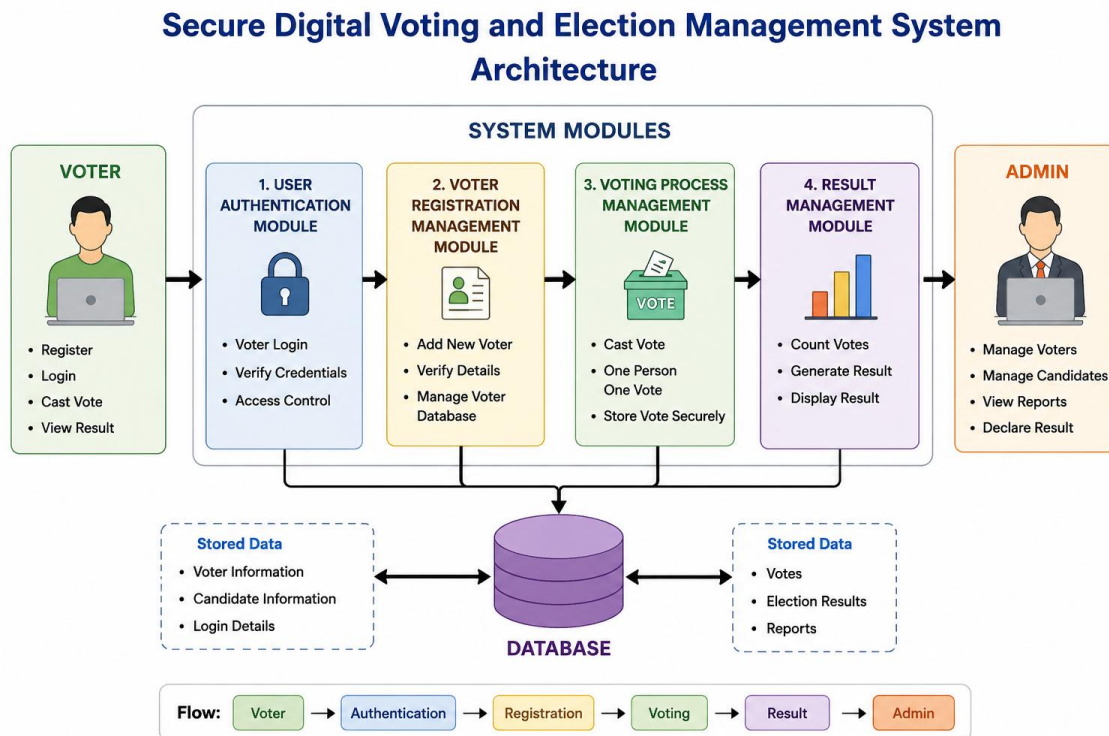
APPENDIX A - DETAILED IMPLEMENTATION LOGIC

The following pseudocode summarizes the engineering sequence used by the prototype. It is included to make the implementation decisions traceable without reproducing an entire application repository.

Registration Flow

1. Receive voter ID, password, name and age.
2. Check age ≥ 18 .
3. Check voter ID uniqueness.
4. Create Voter object from User base attributes.
5. Persist voter record using fstream.
6. Return registration success response.

APPENDIX B - ENGINEERING DRAWINGS / SCHEMATICS



Appendix Figure B.1 Architecture diagram used for implementation planning

APPENDIX C - SOURCE CODE / CODE REPOSITORY INFORMATION

The full source repository is maintained separately. The excerpts below provide representative C++ implementation logic aligned with the modules and route behavior documented in the supplied project materials.

C.1 User and Voter classes

```
class User {
protected:
    std::string id, name, password;
public:
    User(std::string i, std::string n, std::string p)
        : id(std::move(i)), name(std::move(n)), password(std::move(p)) {}
    const std::string& getId() const { return id; }
    bool checkPassword(const std::string& p) const { return p == password; }
};

class Voter : public User {
```

```

    int age;
    bool hasVoted;
public:
    Voter(std::string i, std::string n, std::string p, int a)
        : User(std::move(i), std::move(n), std::move(p)), age(a), hasVoted(false) {}
    int getAge() const { return age; }
    bool alreadyVoted() const { return hasVoted; }
    void markVoted() { hasVoted = true; }
};

```

C.2 Authentication manager

```

class AuthManager {
    std::vector<Voter> voters;
public:
    Voter* authenticate(const std::string& voterId, const std::string& password) {
        for (auto& voter : voters) {
            if (voter.getId() == voterId && voter.checkPassword(password))
                return &voter;
        }
        return nullptr;
    }

    bool registerUser(const std::string& id, const std::string& name,
                     const std::string& password, int age) {
        if (age < 18) return false;
        for (const auto& v : voters)
            if (v.getId() == id) return false;
        voters.emplace_back(id, name, password, age);
        return true;
    }
};

```

C.3 Candidate tally and duplicate-vote guard

```

struct Candidate {
    int id;
    std::string name;
    int voteCount = 0;
};

class ElectionManager {
    std::vector<Candidate> candidates;
public:
    bool castVote(Voter& voter, int candidateId) {
        if (voter.alreadyVoted()) return false;
        for (auto& candidate : candidates) {
            if (candidate.id == candidateId) {
                ++candidate.voteCount;
                voter.markVoted();
                return true;
            }
        }
        return false;
    }
};

```

C.4 File persistence with fstream

```

void saveVoter(const Voter& voter, const std::string& fileName) {
    std::ofstream out(fileName, std::ios::app);
    if (!out) throw std::runtime_error("Unable to open voter file");
}

```

```

SIMATS ENGINEERING - CAPSTONE PROJECT REPORT
// Store the fields required for reconstruction of the voter record.
// Production versions should use structured storage and secure hashing.
}

```

APPENDIX D - DATASHEETS

No external hardware datasheets are required for this software-only prototype. The implementation uses standard computing equipment and software tools documented in Chapter 4.

APPENDIX E - EXPERIMENTAL / RAW DATA

Functional test evidence is summarized in Table 4.2. The raw test inputs consist of valid and invalid voter registration attempts, valid and invalid login attempts, first vote submissions, and repeated vote submissions for the same authenticated voter.

APPENDIX F - ETHICS / INSTITUTIONAL APPROVAL

This prototype is presented as an academic software engineering project. No personal voter dataset beyond demonstration records shown in the project materials is required for the implemented prototype.

APPENDIX G - PROJECT MEETING / PROGRESS RECORDS

Milestone	Planned Activity	Evidence / Status
1	Problem definition and module planning	Completed
2	C++ class and authentication implementation	Completed
3	Registration and persistence implementation	Completed
4	Voting workflow and duplicate-vote testing	Completed
5	Integration, screenshots and documentation	Completed